Hangman in Rust

COMP10068 Secure Programming – Assignment 2 Report

How I approached the problem

The starting template was just a single main function printing Hello world, so I had to build everything on top of it. I split the program into two functions: main handles the play-again loop between rounds, and play_game holds all the game logic for a single round. I found this made the code much easier to manage because each round starts with its own clean state. I also had to update Cargo.toml to include the rand crate as a dependency since Rust does not include random number generation in its standard library.

Reading the fruit list and removing duplicates

I used fs::read_to_string to load fruits.txt at the start of every round (The Rust Project Developers, 2024b). I noticed the file had a number of repeated entries, Banana appears ten times and Blood orange appears twice for example. Rather than editing the file, I collected the lines into a HashSet which drops duplicates automatically since it only holds unique values (The Rust Project Developers, 2024a). Here is that part of the code:

```
let unique: HashSet<&str> = contents.lines()
    .map(|l| l.trim())
    .filter(|l| !l.is_empty())
    .collect();
let secret: &str = fruit_vec.choose(&mut rng).unwrap();
```

I then converted the HashSet into a Vec and used choose from rand to pick a random fruit.

Showing the hidden word

To build the display I went through each character in the secret word and replaced any letter with an underscore, but left spaces and apostrophes visible straight away. I thought this was important because the fruit list includes names like Blood orange, which has a space, and Buddha's hand, which has an apostrophe. If those characters were hidden the player would have no way of knowing they needed to guess around them. So from the very first turn the shape of the word is clear with only the actual letters hidden. The display prints with no gaps between characters, so Blood orange would appear as _____ ______ and Buddha's hand would appear as _______'_ ____ until letters are revealed.

Handling guesses

Every guess gets converted to lowercase immediately so the player can type in any case. I check the length of the input to decide whether it is a single letter or a whole word attempt. For a letter I find every matching position in the secret and reveal them all. A wrong letter costs one attempt. I also penalise repeated guesses so the player cannot just keep entering the same letter, which I tested and confirmed works correctly. For a full word guess I compare it directly against the secret. If it is wrong I deduct an attempt and tell the

player how many of their letters actually appear in the word as a small hint.

Winning, losing, and playing again

I set the limit at six wrong guesses to match the classic six-part Hangman figure. At the top of each loop I check whether any underscores are left and whether the counter has hit zero. No underscores means the player wins and I print a success message. Zero guesses means they lose and I reveal the secret word. I ran several test games to confirm both outcomes worked correctly, including the repeated guess penalty and the whole word guess hint. After each round the outer loop in main asks whether to play again. Pressing Enter or typing y starts a fresh round with a newly chosen fruit, and typing n exits the program cleanly.

References

The Rust Project Developers (2024a) *std::collections::HashSet - Rust standard library*. Available at: https://doc.rust-lang.org/std/collections/struct.HashSet.html (Accessed: 28 March 2026).

The Rust Project Developers (2024b) *std::fs::read_to_string - Rust standard library*. Available at: https://doc.rust-lang.org/std/fs/fn.read_to_string.html (Accessed: 28 March 2026).

The Rust Project Developers (2024c) *The Rust programming language*. Available at: https://doc.rust-lang.org/book/ (Accessed: 28 March 2026).

The Rust Project Developers (2024d) *std::io - Rust standard library*. Available at: https://doc.rust-lang.org/std/io/index.html (Accessed: 28 March 2026).